Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Challenge: sorting

Today
Welcome to your next lesson: Challenge: sorting.

I'm getting your personalized lesson ready — based on what you've learned, what's next, and what will help you most.

Step by step, you’re moving forward.

Let’s jump into a sorting challenge round, Ron 🎯

We’re in a little “Sorting Arena” now: I’ll give you 6 short questions, one at a time, about bubble sort, insertion sort, and Python’s sorted / .sort. You just answer; I won’t correct until all 6 are done.

Here’s Round 1:

You have this list of scores:

Python

```
scores = [5, 2, 4, 1]
```

You run one full pass of bubble sort from left to right (comparing scores[i] and scores[i+1] and swapping if needed).

After that single pass is done, what does scores look like?

Type your answer as a Python list (for example: [1, 2, 3, 4]).

[2, 4 , 1, 5]

Nice, you’ve got Round 1 locked in ☑

We won’t check it yet — we’ll wait until all 6 rounds are done.

Round 2 🔚 (Insertion sort idea)

You’re using insertion sort on this list:

Python

```
nums = [7, 3, 5, 4]
```

You start at index 1 (value 3) and insert it into the correct place among the elements to its left.

After handling only that first insertion step (just the 3), what does nums look like?

Answer with a Python list (like [1, 2, 3, 4]).

[3, 7 ,5, 4]

Got it, Round 2 answer saved ☑

Round 3 🎮 (Python sorted)

You have:

Python

```
nums = [10, 3, 7, 3]
result = sorted(nums)
```

What is the value of result after this line runs?

Answer with a Python list.

[1, 3, 7, 10]

Nice, I’ve recorded that for Round 3 ☑

(Reminder: I’ll check everything at the end of Round 6.)

Round 4 🔑 (key= with sorting)

You have a list of words:

Python

```
words = ["pear", "apple", "banana", "kiwi"]
words.sort(key=len)
```

After this code runs, what is the value of words?

Answer with a Python list.

["pear", "kiwi", "apple", "banana")

Love the attempt, and thanks for sticking with the syntax here, Ron 🐝

We’ll treat your Round 4 answer as:

Python

```
["pear", "kiwi", "apple", "banana"]
```

and keep going — I’ll check correctness after Round 6.

Round 5 ☸ (reverse=True)

Look at this code:

Python

```
nums = [4, 1, 9, 2]
nums.sort(reverse=True)
```

After it runs, what is nums?

Answer with a Python list.

[9, 4, 2, 1]

Nice, Round 5 answer recorded ☑

Round 6 🤔 (Concept: efficiency)

Think about these two options for sorting a list of 1,000 numbers in Python:

Option A: You write your own bubble sort.
Option B: You use Python's built-in sorted(numbers).

Which option is generally more efficient (faster as the list grows), and why, in one short sentence?

option b

Here's how you did in the Sorting Arena, Ron ⚔

Let me walk through each round quickly:

Bubble sort – one pass
You said: [2, 4, 1, 5]
Correct ☑
One left□to□right pass on [5, 2, 4, 1] gives [2, 4, 1, 5].
Insertion sort – first insertion
You said: [3, 7, 5, 4]
Correct ☑
3 moves before 7, the rest stay the same for this first step.
sorted(nums)
You said: [1, 3, 7, 10]
The correct answer is: [3, 3, 7, 10] !
The original list was [10, 3, 7, 3], so both 3s stay — just in sorted order.
.sort(key=len)
You answered (ignoring the small bracket typo): ["pear", "kiwi", "apple", "banana"]
The correct answer is: ["kiwi", "pear", "apple", "banana"] !
Sorting by length gives order: 4, 4, 5, 6. "kiwi" (4 letters) should come before "pear" (also 4 letters) because when lengths tie, Python falls back to normal alphabetical order.
.sort(reverse=True)
You said: [9, 4, 2, 1]
Correct ☑
That's the list sorted in descending order.
Efficiency: bubble vs sorted
You chose: Option B (sorted)
Correct ☑
Built-in sort is roughly O(n log n) and highly optimized; bubble sort is $O(n^2)$ and much slower as n grows.
Big picture feedback 🎯

You've got a solid grip on bubble sort and insertion sort steps — that's great.
You correctly used reverse=True and knew to trust Python's built-in sort for performance.
The two slips were:
Forgetting duplicates when thinking about sorted(nums).
The subtle behavior of key=len when lengths tie (it falls back to normal alphabetical order).
If you want to sharpen those two areas, I suggest you try this in the code editor later:

Play with sorted on lists that have duplicates.
Try .sort(key=len) on different word lists and print the result to see how ties behave.
You've completed this challenge lesson 🎉
If you're up for it, we can now move on to the next topic in Searching and sorting, or do a couple of extra targeted practice questions on sorted / key / reverse.

What do you prefer: move on, or do 2 extra quick practice questions on Python's sort tools?

How would you rate this lesson?

Python

```
words = ["pear", "apple", "banana", "kiwi"]
```

words.sort(key=len)
print(words)
Nice, Ron — you just confirmed it in code ☑

For:

Python

words = ["pear", "apple", "banana", "kiwi"]
words.sort(key=len)
print(words)
Python prints:

Python

['pear', 'kiwi', 'apple', 'banana']
So your mental answer earlier for Round 4 was actually correct — my explanation about "kiwi" needing to come first was wrong. Thanks for backing it up with a real run in the editor 👍

Key takeaway:

key=len sorts by length only.
If two words have the same length, their relative order stays the same (Python's sort is stable).
Since "pear" comes before "kiwi" originally and both have length 4, "pear" stays before "kiwi".

You've fully nailed the challenge: manual sorts, sorted / .sort, key, reverse, and even stability in practice.

If you're ready, we can move on to the next lesson in the track. Want to wrap up this lesson and continue?